Option / Maybe

son: 6 · FP Foundations · Category: Functional · Level: Intermediate · Build with Wa

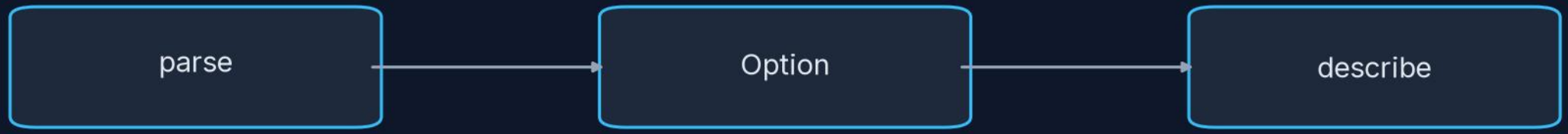
Today's Hook

- Today: Option / Maybe as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Option / Maybe structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Option / Maybe
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/option/ZazrOptionDemo.java
package com.buildwithwaqar.patterns.functional.option;

import dev.zazr.control.Option;

import static dev.zazr.control.Option.none;
import static dev.zazr.control.Option.some;

/** Option / Maybe with Zazr — sealed Some/None, pattern-matchable. */
public final class ZazrOptionDemo {
    private ZazrOptionDemo() {}

    public static Option<Integer> parsePositive(String raw) {
        try {
            int n = Integer.parseInt(raw);
            return n > 0 ? some(n) : none();
        } catch (NumberFormatException e) {
            return none();
        }
    }

    public static String describe(Option<Integer> opt) {
        return switch (opt) {
            case Option.Some(var n) -> "got " + n;
            case Option.None() -> "missing";
        };
    }
```

Implementation (2)

```
    public static Option<Integer> sum3(Option<Integer> a, Option<Integer> b, Option<Integer> c) {
        return Option.zipWith(a, b, c, (x, y, z) -> x + y + z);
    }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.functional.option;
import dev.zazr.control.Option;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Option walkthrough ===");
    System.out.println(ZazrOptionDemo.describe(ZazrOptionDemo.parsePositive("3")));
    System.out.println(ZazrOptionDemo.describe(ZazrOptionDemo.parsePositive("-1")));
    System.out.println("sum=" + ZazrOptionDemo.sum3(Option.some(1), Option.some(2), Option.some(3)));
    System.out.println("sumNone=" + ZazrOptionDemo.sum3(Option.some(1), Option.none(), Option.some(3)));
  }
}
```

Walkthrough Output

```
=== Option walkthrough ===
got 3
missing
sum=Some(6)
sumNone=None
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Option / Maybe
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Option / Maybe → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.